Canary Release

Laboratorio guiado con Nginx, Docker Compose y GitHub Actions

Material docente y guía de prácticas | Nivel licenciatura

Índice

1. Diseño instruccional **1**

2. Fundamentos: por qué surge Canary **1**
2.1. Escenarios donde resulta apropiado 1
2.2. Proceso principal 2

3. Demostración corta: Nginx ponderado **2**

4. Laboratorio: aplicación y arquitectura **2**
4.1. Preparación en la VM Vagrant 3
4.2. Etapa 1: línea base (Blue 100 %) 3
4.3. Etapa 2: Canary al 10 % 3
4.4. Etapa 3: promoción gradual 3
4.5. Etapa 4: inyección de fallos y rollback 3

5. Automatización con GitHub Actions **4**

6. Evidencias y evaluación **4**

7. Preguntas de cierre **4**

8. Referencias y recursos **5**

1. Diseño instruccional

Duración sugerida: 150 minutos (puede dividirse en dos sesiones). **Prerequisitos:** VM Vagrant con Docker Engine y plugin Compose, terminal Linux, Nginx como proxy inverso, repositorio GitHub y nociones de workflows. No requiere Kubernetes.

Resultado de aprendizaje: el estudiante configura y evalúa una liberación Canary de una API HTTP, controla la proporción de tráfico con Nginx, reconoce un fallo funcional y ejecuta un rollback justificando la decisión con evidencias.

Momento	Minutos	Actividad y evidencia
Activación	15	Comparar Recreate y Blue-Green; plantear riesgo de activar v2 para todos.
Fundamentos	25	Definición, origen del patrón, criterios de uso y proceso de promoción.
Demo Nginx	25	Configurar upstream 9:1; observar 100 solicitudes y discutir variabilidad.
Laboratorio	55	Ejecutar API v1/v2, 90/10, 50/50, 100/0 y reversión.
Incidente	20	Inyectar error en v2; interpretar fallos y ejecutar rollback.

Cierre	10	Evidencias, reflexión y ticket de salida.

2. Fundamentos: por qué surge Canary

En un despliegue de tipo *Recreate*, la versión anterior deja de prestar servicio antes de que la nueva esté lista, lo que puede producir indisponibilidad. En *Blue-Green*, ambas versiones coexisten y un cambio de enrutamiento puede enviar prácticamente todo el tráfico a la nueva versión. Esto facilita la reversión, pero el cambio puede exponer de inmediato a toda la población de usuarios a un defecto no detectado.

Canary Release reduce el radio de impacto inicial: se libera la versión nueva a una porción pequeña del tráfico, se observa su comportamiento y, si cumple criterios previamente definidos, se incrementa la exposición. Su nombre alude al uso histórico de canarios como señal temprana de peligro en minas; en software es una analogía de detección temprana.

2.1. Escenarios donde resulta apropiado

- Servicios web con tráfico suficiente para observar diferencias entre versiones.
- Cambios de lógica de negocio de riesgo moderado/alto, especialmente si hay métricas de error y latencia.
- Equipos que necesitan reducir el alcance de una regresión y cuentan con reversión rápida.
- APIs que pueden operar con versiones simultáneas y contratos compatibles.

No es ideal por sí solo cuando existen cambios incompatibles de base de datos, efectos secundarios irreversibles, muy poco tráfico para obtener evidencia o sesiones que requieren afinidad y no la tienen.

2.2. Proceso principal

1. Preparar v2 y probarla sin tráfico público.
2. Definir métricas y umbrales de aceptación (por ejemplo, porcentaje de respuestas 5xx y latencia).
3. Activar una exposición inicial, por ejemplo 10 % de solicitudes.
4. Observar y comparar v1/v2 durante una ventana definida.
5. Si cumple criterios, aumentar 10 % $\rightarrow$ 50 % $\rightarrow$ 100 %; si falla, regresar a 0 %.
6. Registrar decisión, evidencias y resultados.

Idea clave. Un peso 9:1 en Nginx expresa una proporción aproximada de solicitudes, no una asignación persistente del 10 % de usuarios. No garantiza 90/10 exacto en muestras pequeñas.

3. Demostración corta: Nginx ponderado

Pregunta detonadora: si Blue (v1) y Green (v2) están ejecutándose, ¿cómo enviamos solo una fracción de solicitudes a Green?

```
upstream backend {
    server blue:3000 weight=9;
    server green:3000 weight=1;
}
server {
    listen 80;
    location / { proxy_pass http://backend; }
}
```

Nginx usa *weighted round-robin* como mecanismo de selección predeterminado en el grupo upstream. Para el laboratorio completo se incluye una configuración más detallada con cabeceras HTTP. Los nombres `blue` y `green` se resuelven mediante el DNS de la red interna de Compose.

Predicción: ¿cuántas respuestas de Green esperas en 100 solicitudes? ¿Y en 10? La proporción observada puede variar; el laboratorio busca una tendencia, no un conteo exacto.

4. Laboratorio: aplicación y arquitectura

Se proporciona una API Node.js con tres rutas: `/health` (salud básica), `/api/info` (identifica versión) y `/api/checkout` (operación ficticia de negocio). Se construyen dos imágenes desde el mismo código, con `RELEASE_VERSION=v1` y `v2`. Para simular un defecto funcional, Green puede recibir `SIMULATE_ERROR=true`: su `/health` seguirá devolviendo 200, pero `/api/checkout` devolverá 503.

Idea clave. La simulación utiliza el mismo código con un identificador de versión y un fallo configurable. En un release real, v1 y v2 normalmente provienen de revisiones distintas del código y artefactos inmutables diferentes.

4.1. Preparación en la VM Vagrant

Descomprimir el proyecto y situarse en el directorio `canary_lab`. Verificar `docker -version`, `docker compose version`, `curl -version` y disponibilidad del puerto 8080. Si el proyecto se encuentra en un directorio sincronizado por Vagrant, comprobar que los permisos permiten escribir en `nginx/active.conf`.

```
cp .env.example .env
docker compose up -d --build
docker compose ps
curl -s http://127.0.0.1:8080/api/info
```

La salida inicial debe contener `"version":"v1"`. Solo el servicio `proxy` publica el puerto 8080; Blue y Green están dentro de la red de Compose. Si se accede desde Windows, utilizar la IP accesible de Vagrant o configurar `forwarded_port`.

4.2. Etapa 1: línea base (Blue 100 %)

```
./scripts/promote.sh blue
./scripts/sample.sh 100
```

Registrar la distribución y verificar que Green existe pero no recibe tráfico público. Inspeccionar `nginx/active.conf` y localizar la instrucción `proxy_pass` y el bloque `upstream`.

4.3. Etapa 2: Canary al 10 %

```
./scripts/promote.sh canary10
./scripts/sample.sh 100
```

Registrar conteos de v1 y v2. Explicar por qué el peso 9:1 no equivale a una probabilidad garantizada para cada grupo de diez solicitudes ni a 10 % de usuarios únicos.

4.4. Etapa 3: promoción gradual

```
./scripts/promote.sh canary50
./scripts/sample.sh 100
./scripts/promote.sh green
./scripts/sample.sh 30
```

Explicar qué cambia entre una liberación Canary y un cambio Blue-Green directo. ¿Qué métrica consultarías antes de pasar del 10 % al 50 %?

4.5. Etapa 4: inyección de fallos y rollback

Restaurar Blue y recrear Green con el fallo simulado:

```
./scripts/promote.sh blue
GREEN_FAIL=true docker compose up -d --force-recreate green
./scripts/promote.sh canary10
./scripts/check-checkout.sh 50
```

Algunas solicitudes pueden devolver `503`, aunque `/health` de Green siga indicando salud. Registrar cuántas respuestas 200/503 aparecieron y decidir si se debe promocionar.

```
./scripts/promote.sh blue
./scripts/check-checkout.sh 20
GREEN_FAIL=false docker compose up -d --force-recreate green
```

Discusión: ¿por qué el health check no detectó el problema? ¿Qué consecuencias tendría mantener la promoción? ¿Qué pasaría si una migración de base de datos de v2 fuese incompatible con v1?

5. Automatización con GitHub Actions

El repositorio incluye `.github/workflows/canary-ci.yml`. Al hacer push o pull request, un runner de GitHub ejecuta pruebas unitarias con Node, construye las imágenes, inicia Compose, espera una respuesta de salud, valida la distribución 90/10 (presencia de ambas versiones), promociona a Green y revierte a Blue.

Importante: el workflow *no despliega a la VM Vagrant del estudiante*. Un runner hospedado por GitHub normalmente no puede conectarse a una VM privada local. El objetivo de esta actividad es automatizar la *validación de la estrategia* en un entorno temporal. Un CD hacia la VM requeriría conectividad, autenticación SSH segura, controles de acceso y aprobación del ambiente.

Actividad de inspección: identificar `on`, `jobs`, `runs-on`, `steps`, `uses` y `run`. Indicar qué evidencia proporciona cada paso y qué pasos no deberían omitirse antes de promover una versión.

6. Evidencias y evaluación

Criterio	Puntos
Explica origen, utilidad y diferencia con Blue-Green	15
Muestra Compose y Nginx funcionales, con ambas versiones	20
Registra distribución 100/0, 90/10, 50/50 y 0/100	20
Detecta fallo funcional, argumenta y ejecuta rollback	25
Explica workflow y sus límites respecto a Vagrant	10
Reporte claro, reproducible y con reflexión técnica	10

Entregable: repositorio GitHub con código y workflow, archivo README con comandos, capturas o salidas de las cuatro etapas, resultados del incidente y reflexión de una página sobre criterios de promoción. No incluir credenciales ni llaves privadas.

7. Preguntas de cierre

1. ¿Qué diferencia existe entre distribuir el 10 % de solicitudes y liberar a un 10 % fijo de usuarios?
2. ¿Por qué una respuesta 200 en `/health` no basta para aprobar una versión?
3. ¿Qué aporta Canary frente a Blue-Green y qué costo adicional introduce?
4. ¿Qué estrategia aplicarías si la versión nueva modifica de forma irreversible el esquema de datos?
5. ¿Por qué GitHub Actions puede validar Compose pero no necesariamente desplegar en tu VM Vagrant?

8. Referencias y recursos

- NGINX. *HTTP Load Balancing*. https://docs.nginx.com/nginx/admin-guide/load-balancer/http-load-balancer/
- Docker. *Docker Compose documentation*. https://docs.docker.com/compose/
- GitHub. *Understanding GitHub Actions*. https://docs.github.com/en/actions/about-github-actions/understanding-github-actions
- Google. *Site Reliability Engineering: Release Engineering*. https://sre.google/sre-book/release-engineerin
- Humble, J., y Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.